Electronics Club, IIT Guwahati · VLSI Track · Inter IIT Bootcamp

Digital Decimation Filter for a ΣΔ ADC

CIC decimator and CIC + FIR droop compensator, OSR 2048, 20-bit output

Rahul Jain · B.Tech ECE, IIT Guwahati · 15 October 2026

Summary. Two synthesizable Verilog decimators were designed for the ΣΔ bit stream supplied with the problem statement (4.096 MHz, OSR 2048). Both produce a 20-bit word at 2 kHz. **Task 1** is a third-order CIC that decimates by 2048. **Task 2** is a fourth-order CIC that decimates by 512, followed by a 27-tap linear-phase FIR that both corrects the CIC droop and decimates by 4. The FIR uses one shared multiplier. Scored with the official `fft_2.m` on Vivado simulation outputs, the SNDR is **120.68 / 118.61 / 110.41 dB** for Task 1 and **121.20 / 117.66 / 118.57 dB** for Task 2 at 10 / 500 / 990 Hz. No output sample was clipped. At the band edge the compensator recovers **+8.2 dB**. In steady state both chains sit within about 1 dB of the ideal 20-bit limit (about 122 dB).

1 Objective

Design the digital back-end of a ΣΔ ADC. It must low-pass filter the oversampled modulator output and decimate it by the oversampling ratio (OSR = 2048). The goal is to maximise the output signal-to-noise-and-distortion ratio (SNDR) with a 20-bit output word.

- **Task 1:** a standalone CIC decimation filter.
- **Task 2:** a CIC decimator followed by an FIR filter that compensates the CIC passband droop.

Deliverables are synthesizable Verilog, PSD plots from the provided MATLAB script, and a justification of the coefficients chosen.

2 Specification and provided material

Item	Value / source
Input	1-bit ΣΔ stream, files `hz10.mem`, `hz500.mem`, `hz990.mem` (4 096 026 bits each)
Testbench mapping	`tb.v`: even index → `{0,d}` $= +d$, odd index → `{d,d}` $= -d$; so `bits` $\in \{-1, 0, +1\}$ (2-bit signed)
Output rate f_{out}	2 kHz (`Fs = 2e3` in `fft_2.m`); signal band 0–1 kHz
Input rate f_s	$f_{out} \times$ OSR $= 2\,\text{kHz} \times 2048 = 4.096\,\text{MHz}$
Record length	4 096 026 bits ≈ 1 s $\rightarrow$ 2000 output samples, 1 Hz FFT bins (coherent tones)
Output word	20-bit signed (full scale $\pm 524\,287$)
Scoring	`fft_2.m`: last 2000 samples, periodic Hann window, signal = 3 bins, noise = rest up to 1 kHz
Simulation clock	100 ns per input bit (simulation time only; the real rate is 4.096 MHz)

3 Input signal analysis

Before choosing the filter order, the PSD of the provided bit stream was computed (Fig. 1). Three properties drive the design:

1. **The tone is small.** Its amplitude is 0.278 of full scale (-11.1 dBFS). The output word, not the modulator, therefore limits the SNDR, so a fixed gain of $\times 3.5$ is applied to use the full 20-bit range (Sections 5.4 and 6.3).
2. **The quantisation noise is steeply shaped.** It is very low in band and rises by roughly 100 dB between 3 kHz and 1 MHz, which is characteristic of a high-order modulator. Every band that folds onto 0–1 kHz during decimation (around $k \cdot f_{out}$) must be strongly attenuated. A first- or second-order averaging filter is not enough (Table 2).
3. **There is a strong component at $f_s/2$.** It comes from the alternating-sign mapping in the testbench. It sits exactly on a CIC null ($f_s/2 = 1024 f_{out} = 256\, f_s/512$), so both designs remove it completely.

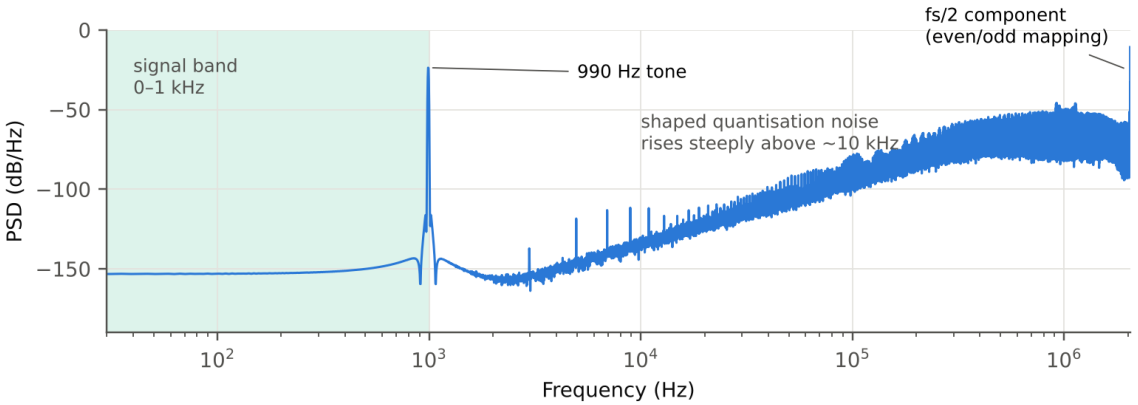


Figure 1: PSD of the 990 Hz input bit stream after the testbench mapping (Welch, Blackman–Harris window, 3.9 Hz resolution).

4 Architecture overview

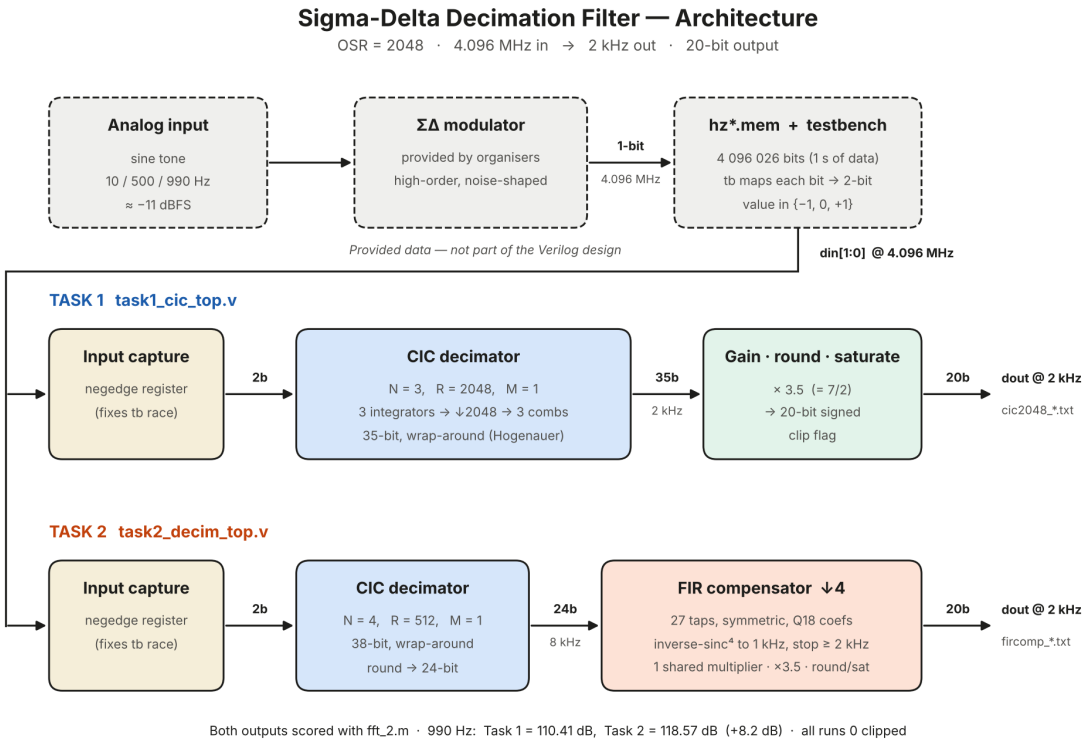


Figure 2: Signal chain of both tasks. The dashed blocks (modulator data and testbench) were provided; the solid blocks are the Verilog design.

Both top levels first register the input bit on the *falling* clock edge (`din_q`). The template testbench updates `bits` with a blocking assignment on the rising edge. That races with any logic sampled on the same edge, and in one simulator the race corrupted the input. Capturing mid-cycle removes the race, so the result is simulator-independent, at the cost of one 2-bit register.

Task	Stage	Rate	Word length	Operation
1	CIC integrators ×3	4.096 MHz	35 bit	running sums, wrap-around
1	CIC combs ×3 + gain	2 kHz	35 → 20 bit	differences, ×7/2, round, saturate
2	CIC integrators ×4	4.096 MHz	38 bit	running sums, wrap-around
2	CIC combs ×4	8 kHz	38 → 24 bit	differences, round
2	FIR (27 taps), ↓ 4	8 kHz in, 2 kHz out	24 → 20 bit	14 MACs on one multiplier, round, saturate

Table 1: Rates and word lengths.

5 Task 1 — standalone CIC decimator

5.1 Transfer function

A CIC filter of order N, decimation R and differential delay M is the cascade of N moving-sum filters of length RM:

$$H(z) = \left(\frac{1 - z^{-RM}}{1 - z^{-1}}\right)^N, \qquad |H(f)| = \left|\frac{\sin(\pi f RM/f_s)}{\sin(\pi f/f_s)}\right|^N. \tag{1}$$

It is built as N integrators at f_s, a down-sampler, and N combs ($1 - z^{-M}$) at f_s/R (Hogenauer). This moves the long delay z^{-RM} to the low rate, where it becomes a single register. The filter needs no multipliers and no coefficient memory. Its nulls fall exactly on $k \cdot f_s/R$, which are the frequencies that alias onto DC.

5.2 Choice of N, R and M

$R = 2048$ is fixed because Task 1 must perform the whole decimation. $M = 1$ is used because $M = 2$ adds droop and N extra bits without improving alias rejection at the folding frequencies. The order N was chosen by simulating the full bit-exact chain on all three vectors (Table 2).

N	B_{max}	Droop @ 990 Hz	10 Hz	500 Hz	990 Hz	Worst tone	Peak $\lvert y\rvert$
2	24	−7.67 dB	113.36	108.70	105.64	105.64	509 686
3	35	−11.51 dB	120.68	118.61	110.41	**110.41**	509 605
4	46	−15.34 dB	121.04	131.73	105.96	105.96	509 643
5	57	−19.18 dB	121.14	125.85	102.44	102.44	509 563

Table 2: Task 1 order sweep. SNDR in dB from `fft_2.m`, bit-exact model at the RTL decimation phase, gain 3.5. No clipping in any case.

Why N = 3.

- $N = 2$ **is too weak.** Its sidelobes let modulator noise alias into the band, costing 7–10 dB at every tone.
- $N \geq 4$ **costs too much at the band edge.** The droop at 990 Hz grows by 3.8 dB per extra stage, and Task 1 has no compensator. The 990 Hz tone is the binding (worst-case) constraint, and $N = 3$ maximises it.
- **The 500 Hz scores for** $N \geq 4$ **are not real gains.** 500 Hz is exactly $f_{out}/4$, so the output repeats every 4 samples and the rounding error becomes a fixed pattern rather than noise (Section 9).

5.3 Register width

The DC gain of the CIC is $(RM)^N$, so the registers need

$$B_{max} = N\lceil \log_2(RM) \rceil + B_{in} = 3 \cdot 11 + 2 = 35 \text{ bits}. \tag{2}$$

The integrators are allowed to overflow, with two's-complement wrap-around. Each comb only forms a difference, and modular arithmetic gives the exact difference whenever the true result fits in B_{max} bits. B_{max} is chosen so that it always does. Saturating the integrators would be wrong, because it would break the cancellation.

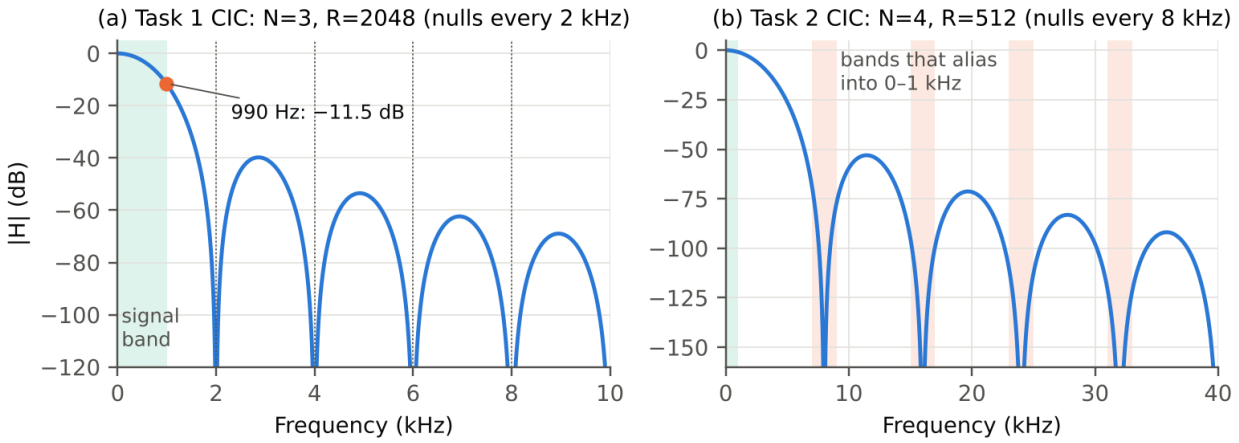


Figure 3: (a) Task 1 CIC: nulls every 2 kHz remove all aliasing bands, but the 990 Hz tone is attenuated by 11.5 dB. (b) Task 2 CIC: nulls every 8 kHz; the bands that alias into 0–1 kHz after $\downarrow 512$ are shaded.

5.4 Output scaling, rounding and saturation

With gain 1, a full-scale input (± 1) maps to $\pm 2^{33}$. Keeping the top 20 bits is therefore an arithmetic right shift of $33 - 19 = 14$.

The benchmark tone is only -11.1 dBFS, so a constant gain of $3.5 = 7/2$ is applied. It is implemented as a $\times 7$ constant multiply (shift-and-add) followed by a shift of 15. The output then peaks at 509 605, which is -0.25 dBFS, leaving 2.8 % headroom.

Rounding is round-half-up: add 2^{14}, then shift right by 15. Truncation would add a $-\frac{1}{2}$ LSB bias. Values outside $\pm 524\,287$ saturate and pulse `clip`. Saturation is used because wrapping would flip the sign of an over-range sample.

5.5 Implementation (`cic_decimator.v`)

- `integ[0..N-1]`: `integ[0]` accumulates the sign-extended input; `integ[k]` accumulates `integ[k-1]`. All are updated every input clock.
- An 11-bit counter produces a one-clock `tick` every R inputs. On a tick, `cval[0]` samples `integ[N-1]`.
- The combs are sequenced by a shift register `vs`, one stage per clock: `cval[s] = cval[s-1] - dl[s-1]`, after which `dl[s-1]` stores `cval[s-1]`. A single subtractor per stage gives a short critical path, and the latency is only $N + 1$ clocks out of the R available.
- The module is fully parameterised (N, R, M, widths, gain), so Task 2 reuses it unchanged.

6 Task 2 — CIC + FIR droop compensator

6.1 Partitioning: why 512 × 4

A CIC is cheap but has a soft passband and transition. A FIR can be sharp, but running it at 4.096 MHz would be costly. The decimation is therefore split:

- The CIC performs the large, cheap decimation by $R = 512$, down to 8 kHz. At that rate the 990 Hz tone is only at $0.124\, f_s / R$, so the droop is small (-0.88 dB rather than -11.5 dB).
- The FIR then removes the 1–4 kHz modulator noise, flattens the droop and decimates by the final 4.

The template's `wire signed [37:0] cicout` and its `cic512_*_by4.txt` file names point to the same split, since $4 \cdot 9 + 2 = 38$ bits. As a baseline, CIC-512 followed by a plain $\downarrow 4$ (same gain, no FIR) scores only 113.78 / 115.18 / 112.29 dB. Without filtering, the noise between 1 and 4 kHz aliases into the band.

6.2 CIC stage

- $N = 4$, $R = 512$, $M = 1$, giving $B_{max} = 4 \cdot 9 + 2 = 38$ bits with wrap-around.
- The higher order (compared with Task 1) is affordable because the FIR corrects the extra droop, and it gives more alias rejection at 8, 16, 24 kHz … (Fig. 3b).
- The output is rounded to a **24-bit interstage word**: full scale $2^{36} \to 2^{23}$ is a shift of 13. The 20-bit limit applies only to the final bus, and rounding to 20 bits here would add a second rounding-noise source, costing about 2 dB.
- CIC droop at the test tones: 0.000 / −0.224 / −0.880 dB.

6.3 FIR specification and design method

The FIR runs at $f_1 = 8$ kHz and must:

- invert the CIC droop over the signal band, with $|H_{FIR}(f)| = 1/|H_{CIC}(f)|$ for $0 \le f \le 1$ kHz;
- attenuate 2–4 kHz, which would alias after $\downarrow 4$;
- have linear phase (symmetric taps), so the waveform is not distorted and the symmetry can be exploited in hardware.

The coefficients were obtained by weighted least squares (`firls`):

- **Passband target.** The passband was split into 59 sub-bands, each with target $1/|H_{CIC}|$ at its edges, so the inverse-sinc^4 shape is followed closely.
- **Stop band.** Target 0 over 2–4 kHz, with weight 10.
- **Transition band (1–2 kHz).** Left free. This region folds onto 0–1 kHz after $\downarrow 4$, but the modulator noise there is at its minimum (Fig. 1). Table 6 confirms that a sharper transition does not raise the SNDR.

The taps were then normalised to unity DC gain, $h \leftarrow h/\sum h$, and the overall gain was folded in:

$$g = 3.5 \cdot \frac{2^{19}}{2^{23}} = 0.21875, \qquad c[n] = \text{round}\left(h[n] \cdot g \cdot 2^{18}\right). \tag{3}$$

Here $2^{19}/2^{23}$ maps the 24-bit interstage full scale onto the 20-bit output full scale, and 3.5 is the same gain as Task 1. The coefficients are stored as signed Q18 values in 16-bit words. Their sum is 57 345 $\approx 0.21875 \cdot 2^{18}$.

6.4 Chosen coefficients

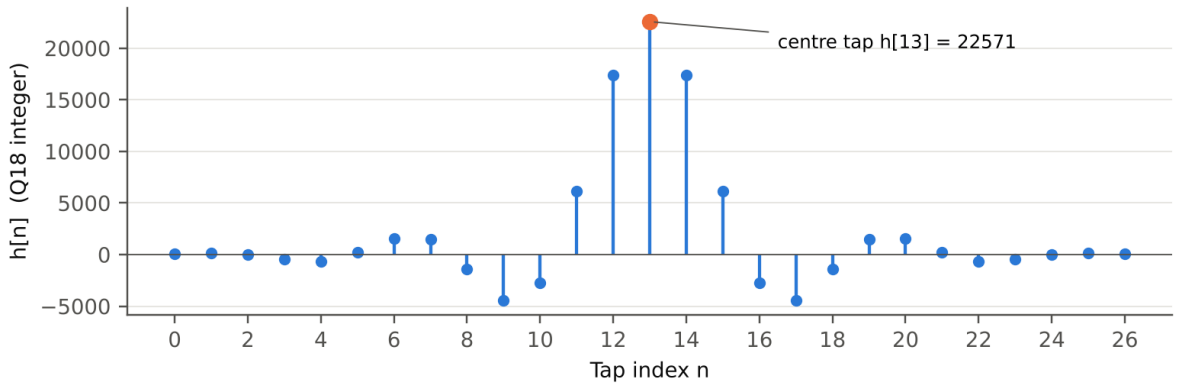


Figure 4: Impulse response (Q18 integers). The main lobe is narrower than a plain low-pass filter, and the negative side taps at $n = 9, 10$ create the high-frequency boost that cancels the CIC droop.

6.5 Justification of the coefficient choices

Every parameter was swept with a bit-exact model of the full chain. That model reproduces the RTL output sample for sample.

Tap n (and mirror $26-n$)	$h[n]$ (unity DC gain)	$c[n]$ (Q18 integer, in RTL)	$c[n]/2^{18}$
0, 26	+0.001310	+75	+0.000286
1, 25	+0.002875	+165	+0.000629
2, 24	+0.000244	+14	+0.000053
3, 23	-0.007795	-447	-0.001705
4, 22	-0.010997	-631	-0.002407
5, 21	+0.003709	+213	+0.000813
6, 20	+0.027616	+1584	+0.006042
7, 19	+0.025232	+1447	+0.005520
8, 18	-0.024340	-1396	-0.005325
9, 17	-0.077990	-4472	-0.017059
10, 16	-0.047159	-2704	-0.010315
11, 15	+0.107113	+6142	+0.023430
12, 14	+0.303376	+17397	+0.066364
13 (centre)	+0.393608	+22571	+0.086102
Sum of all 27 taps		57 345	0.218754

Table 3: The 27 FIR coefficients. The filter is symmetric ($c[n] = c[26-n]$, Type I linear phase), so 14 unique values are stored. The largest, 22 571, fits a signed 16-bit word.

Number of taps. Table 4 lists, for each length K:

- the official score at each tone;
- the passband ripple and the alias rejection;
- a steady-state SNDR, from a sine fit after skipping the first 40 output samples (the start-up transient).

	Official `fft_2.m` SNDR (dB)			Ripple 0–990 Hz	Rejection ≥2 kHz	Steady-state
K	10 Hz	500 Hz	990 Hz	(dB)	(dB)	990 Hz (dB)
15	120.34	121.95	120.62	0.494	35.2	121.5
19	121.21	125.35	120.57	0.395	41.7	121.2
23	119.89	123.75	118.17	0.163	53.1	121.5
27	121.19	117.66	118.57	0.043	58.9	121.7
31	120.80	117.47	114.64	0.018	62.0	121.5
35	120.83	112.97	114.59	0.013	70.9	121.0

Table 4: FIR length sweep (Q18, stop band from 2 kHz, weight 10).

$K = 27$ was selected on the *filter specification*:

- It is the shortest length with passband ripple below 0.05 dB and rejection near 60 dB.
- Shorter filters leave 0.16–0.49 dB of ripple and only 35–53 dB of rejection.
- Longer filters add taps without improving the steady-state SNDR, which stays at 121–122 dB for every K because the 20-bit output word sets it.

The spread in the *official* scores (for example $K = 19$ reads higher at 500 and 990 Hz) is not a filter-quality effect. `fft_2.m` scores all 2000 output samples, including the filter start-up, so a shorter filter settles sooner and reads slightly better. At 500 Hz the score also depends on the fixed rounding pattern at $f_{out}/4$ (Section 9).

A filter tuned to these artefacts would score better on the three test files but would be a worse decimator for general inputs. That trade was deliberately not made.

Coefficient word length. Q12–Q20 were compared (Table 5). From Q14 upwards the response is unchanged (ripple 0.043 dB) and the SNDR varies by less than the run-to-run rounding scatter. Q18 in 16-bit words was chosen: it keeps the coefficient quantisation error far below the 20-bit output LSB and matches a standard 16-bit coefficient ROM width.

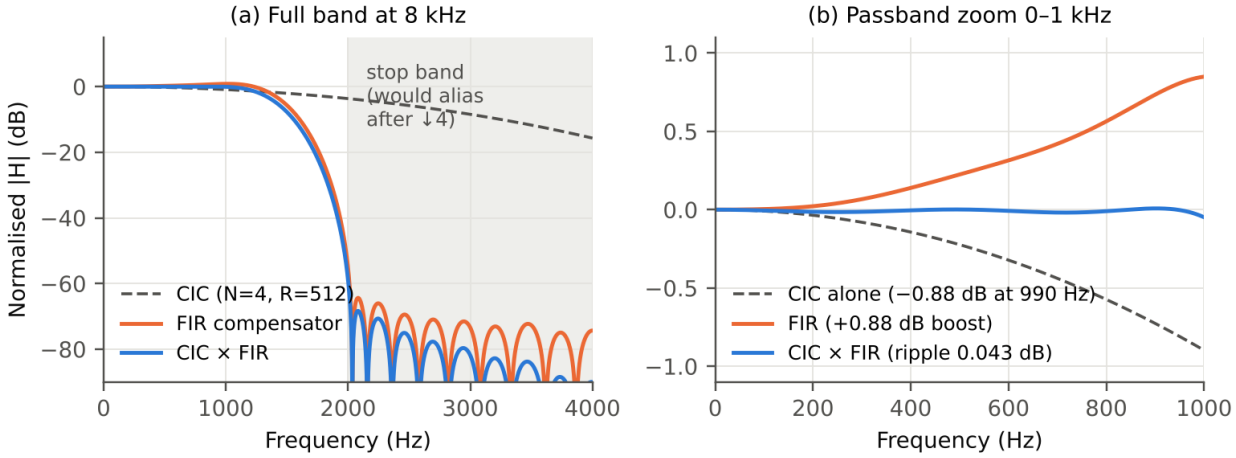


Figure 5: (a) CIC, FIR and combined responses at the 8 kHz rate, giving at least 58.9 dB of attenuation from 2 kHz. (b) Passband detail: the FIR boosts by +0.88 dB at 990 Hz to cancel the CIC droop, and the combined ripple is 0.043 dB over 0–990 Hz.

Format	Width	10 Hz	500 Hz	990 Hz	Ripple	Rej.
Q12	10	120.47	118.72	118.23	0.047	55.7
Q14	12	120.63	119.33	118.87	0.043	60.8
Q16	14	120.70	118.53	118.47	0.043	59.5
Q18	16	121.19	117.66	118.57	0.043	58.9
Q20	18	120.92	117.58	118.26	0.043	59.1

Table 5: Coefficient precision ($K = 27$).

f_{st}	W	10	500	990	Ripple
1500	10	121.30	119.03	118.99	0.922
1750	10	120.16	119.39	118.42	0.258
2000	1	120.74	116.84	118.32	0.036
2000	10	121.19	117.66	118.57	0.043
2000	100	121.45	117.14	117.95	0.077
2500	10	120.00	118.92	117.77	0.001

Table 6: Stop-band edge f_{st} (Hz) and weight W ($K = 27$, SNDR in dB).

Stop-band edge and weight. The trade-offs in Table 6:

- A lower edge (1.5 kHz) forces 0.9 dB of passband ripple.
- A higher edge (2.5 kHz) leaves only about 18 dB of rejection at 2 kHz.
- Within the 2 kHz rows, weight 10 balances ripple (0.043 dB) against rejection (58.9 dB).

6.6 Hardware implementation (`fir_decim_comp.v`)

- **Delay line.** 27 registers of 24 bits, shifted on every CIC output strobe. A 2-bit phase counter raises `start` on every fourth sample, so only the outputs that are kept are computed (decimation is built into the filter).
- **Symmetric pre-adder.** Each cycle forms $p = x[i] + x[26 - i]$ (25 bits; the centre tap $x[13]$ is used alone). This halves the multiplications to 14.
- **One shared multiplier.** An output is due every 2048 input clocks and needs only 14 multiply–accumulate cycles, so one 25×16 multiplier is time-shared. A parallel design would need 14–27 multipliers for no benefit.
- **Pipeline.** Stage 1 registers p and $c[i]$; stage 2 accumulates $p \cdot c[i]$ (41-bit product) into a 48-bit accumulator. The accumulator needs $41 + \lceil \log_2 14 \rceil = 45$ bits; 48 gives margin.
- **Timing.** The 14 products are accumulated on consecutive clocks. The output is registered 17 clocks after the fourth input sample is captured, which is negligible against the 2048-clock output period.
- **Output.** Round half-up ($+2^{17}$, then $\gg 18$) and saturate to 20 bits with a `clip` flag.

7 Verification methodology

1. **RTL simulation.** The behavioural simulation ran in Vivado 2024.2 (xsim) for all six cases (2 tasks × 3 tones). The testbenches are the provided template with the DUT instantiated, the output clock set to $f_s/2048$, and the file names derived from a `TONE` macro. The bit mapping and stimulus are unchanged. Each run produced 2000 output samples with **0 clipped samples** (clip counters in the testbench).
2. **Scoring.** MATLAB R2026b, running the provided `fft_2.m` with `fin` and the file name set per run. The Signal Processing Toolbox was not installed, so `hann(L,'periodic')` was replaced by its definition $0.5 - 0.5\cos(2\pi n/L)$, which gives identical values. The calculation is otherwise unchanged.
3. **Independent cross-checks.**
   - A bit-exact Python model of both chains reproduces all 6 × 2000 Vivado output samples exactly.
   - The same RTL simulated in Icarus Verilog gives identical output files.
   - The sweeps in Tables 2–6 use this verified model.

8 Results

Tone	Task 1: CIC (N=3, R=2048)	Task 2: CIC (N=4, R=512) + FIR	Difference
10 Hz	120.68 dB	121.20 dB	+0.52 dB
500 Hz	118.61 dB	117.66 dB	−0.95 dB
990 Hz	110.41 dB	**118.57 dB**	**+8.16 dB**
Steady-state (sine fit)	120.7 / 118.7 / 110.5	121.2 / 122.4 / 121.7	
Clipped samples	0	0	

Table 7: Final SNDR from the official `fft_2.m` (MATLAB) on the Vivado output files.

Figures 6 and 7 are the MATLAB PSD screenshots produced by `fft_2.m`, one per output file.

9 Discussion

The 20-bit output is the ceiling.

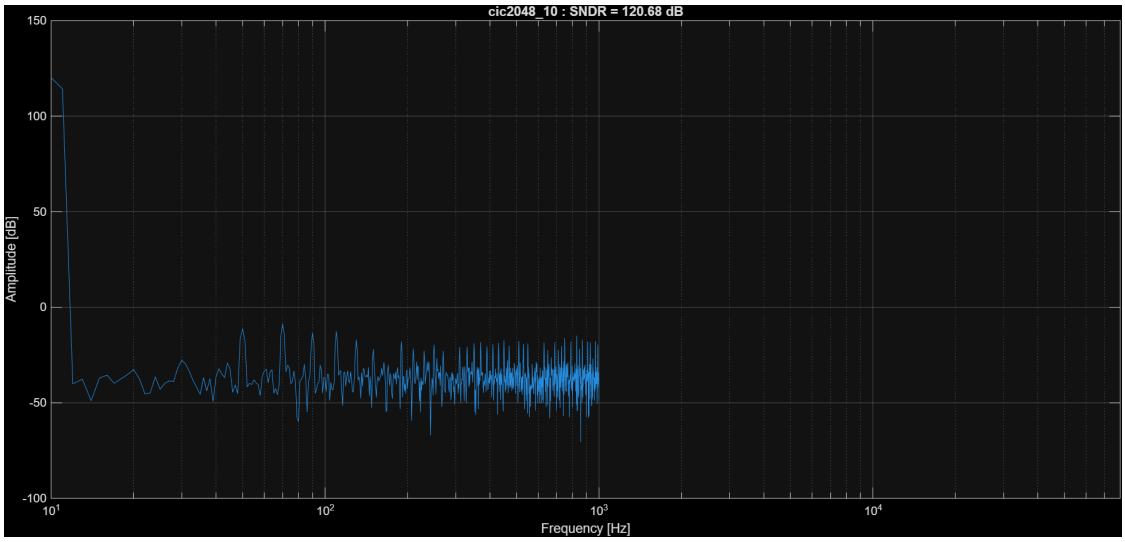
- An ideal 20-bit rounded sine at −0.25 dBFS has $6.02 \cdot 20 + 1.76 - 0.25 \approx 121.9$ dB SNR, and `fft_2.m` returns 121.5–122.8 dB on such a sine at 10 and 990 Hz.
- Without the final rounding, the Task 2 chain reaches 126 dB (10 Hz) and 131–132 dB (500, 990 Hz). This is the noise floor of the modulator and filters themselves.
- The output word therefore dominates, and the measured steady-state values (121.2–122.4 dB) are within about 1 dB of the ideal.
- This is also why the ×3.5 gain matters. Without it, the −11.1 dBFS tone would lose about 11 dB of SNDR to the fixed rounding noise.

Band-edge tone (990 Hz). In Task 1 the CIC attenuates the tone by 11.5 dB (steady amplitude 135 491 instead of 509 700), while the rounding noise stays the same, so the SNDR drops to 110.4 dB. The compensator in Task 2 restores the full amplitude (ripple 0.043 dB). The FIR's high-frequency boost raises the noise near 1 kHz, so the total noise grows by 3.3 dB, and the net improvement is 8.2 dB.

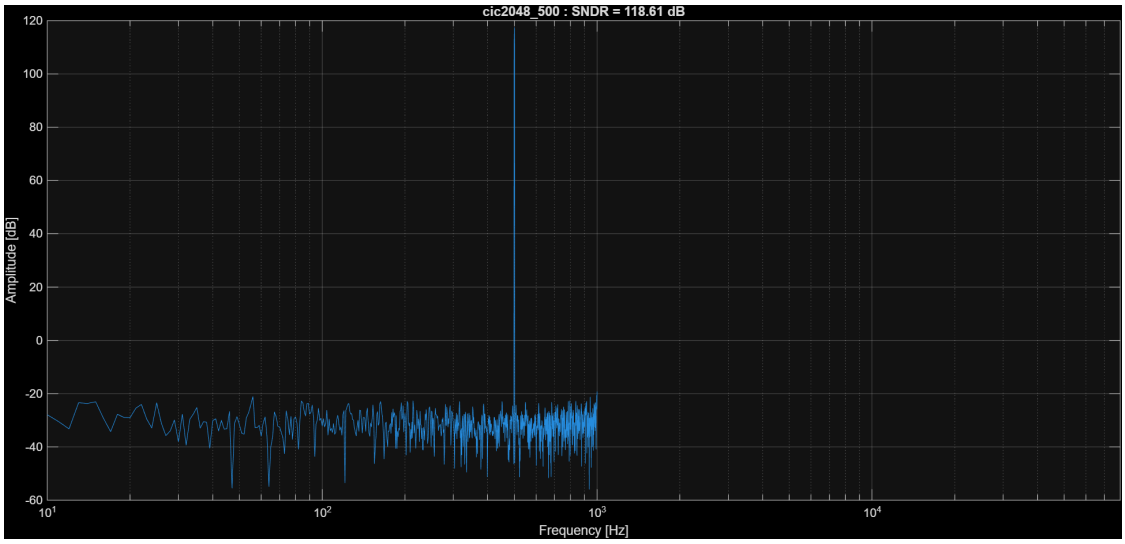
500 Hz behaviour. 500 Hz is exactly $f_{out}/4$, so the output sequence repeats every 4 samples. The rounding error is then a fixed 4-sample pattern rather than white noise, and its power depends on the decimation phase. This explains the large spread of 500 Hz values across Tables 2 and 4.

The slightly lower official Task 2 score at 500 Hz (117.66 vs 118.61 dB) comes from the start-up transient. `fft_2.m` scores all 2000 samples. The Task 2 chain (CIC + 27-tap FIR) needs about 8 output samples to settle, against about 3 for Task 1. Excluding the first 40 samples, Task 2 reaches 122.4 dB at 500 Hz.

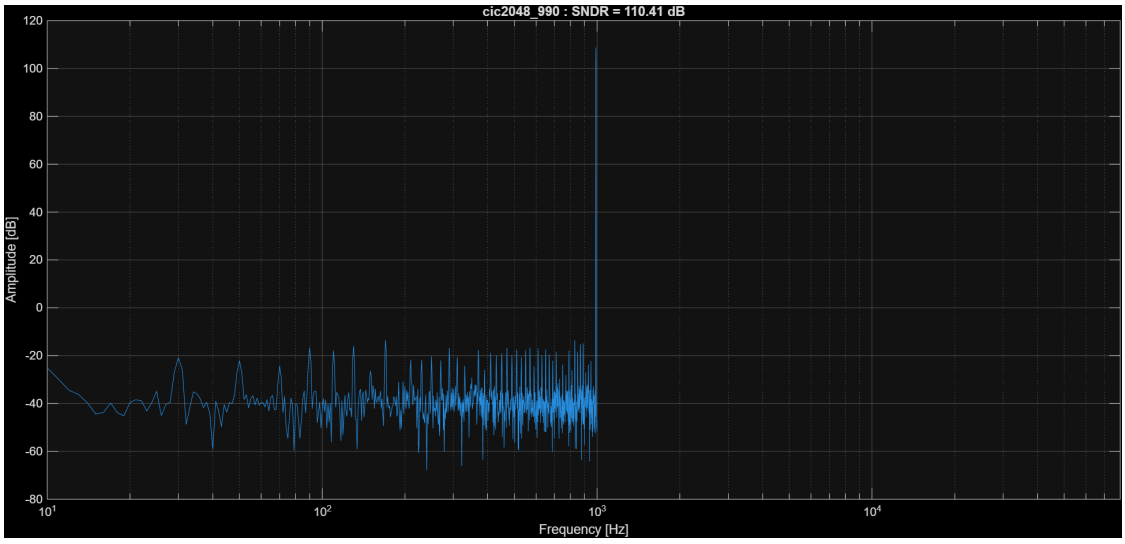
Hardware cost. Both top levels synthesize cleanly with generic synthesis (Yosys):



(a) Task 1, 10 Hz — 120.68 dB

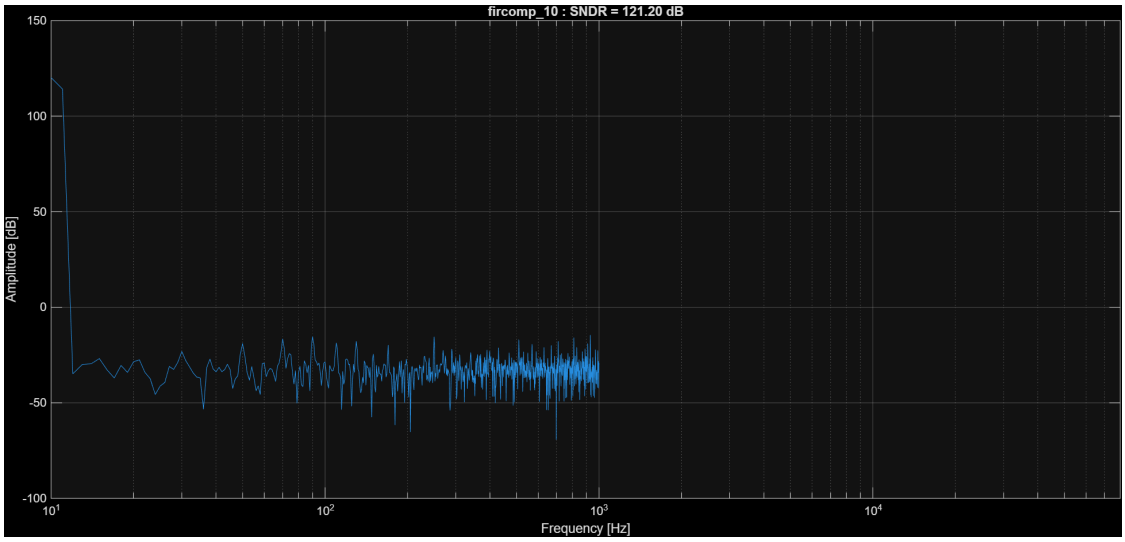


(b) Task 1, 500 Hz — 118.61 dB

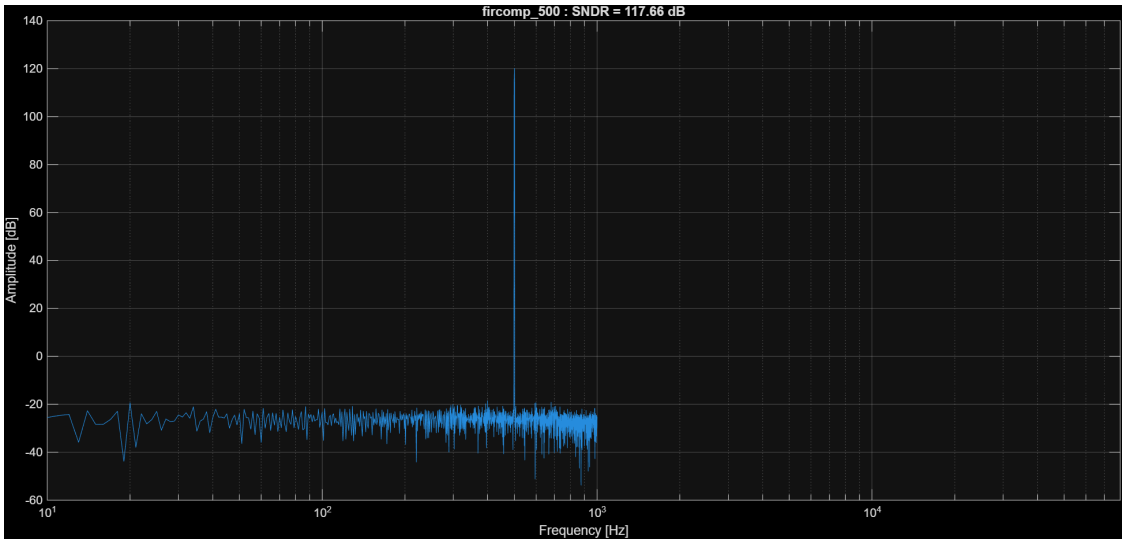


(c) Task 1, 990 Hz — 110.41 dB

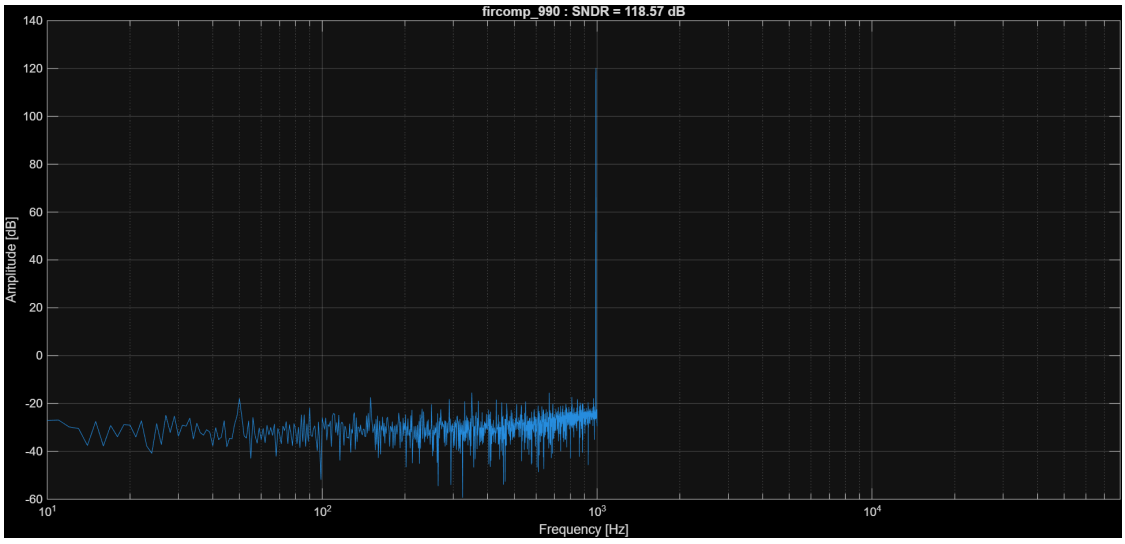
Figure 6: Task 1 (CIC only) output PSD, MATLAB `fft_2.m`. 2000 samples at 2 kHz, Hann window, 1 Hz bins.



(a) Task 2, 10 Hz — 121.20 dB



(b) Task 2, 500 Hz — 117.66 dB



(c) Task 2, 990 Hz — 118.57 dB

Figure 7: Task 2 (CIC + FIR compensator) output PSD, MATLAB `fft_2.m`.

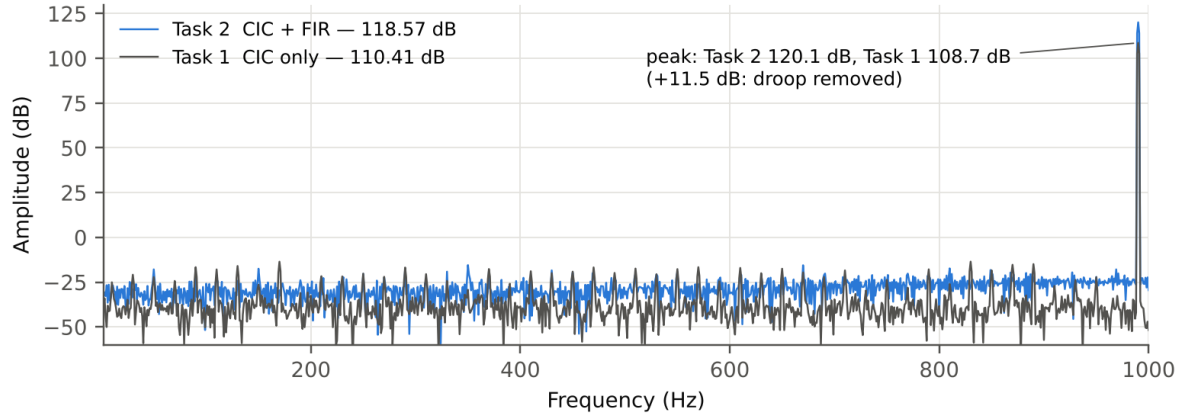


Figure 8: 990 Hz, Task 1 and Task 2 overlaid (same calculation as `fft_2.m`, linear frequency axis). The compensated tone is 11.5 dB higher, while the total noise rises by only 3.3 dB, a net gain of 8.2 dB.

- Task 1 uses about 390 flip-flops and no general multiplier ($\times 7$ is shift-and-add).
- Task 2 uses about 1290 flip-flops, 648 of them in the FIR delay line, and a single 25×16 multiplier.
- The delay line could be mapped to a small RAM to save area.

Limitations. The $\times 3.5$ gain is set for the benchmark input level (-11.1 dBFS). Inputs above about -10.9 dBFS saturate cleanly, never wrap, and raise `clip`. For a general-purpose ADC, gain 1 would be used (`GAIN_NUM=1`, `GAIN_FRAC=0` in Task 1; $G = 1$ in the FIR design), at the cost of about 11 dB on these test vectors.

10 Conclusion

- A third-order CIC ($R = 2048$) meets Task 1 with 120.7 / 118.6 / 110.4 dB. Its band-edge performance is limited by the 11.5 dB CIC droop at 990 Hz.
- Splitting the decimation into a fourth-order CIC ($R = 512$) and a 27-tap inverse-sinc FIR ($\downarrow 4$) flattens the passband to 0.043 dB ripple and rejects aliases by about 59 dB, using a single multiplier. Task 2 reaches 121.2 / 117.7 / 118.6 dB, an 8.2 dB improvement at the band edge.
- In steady state both designs operate within about 1 dB of the theoretical limit of a 20-bit output.
- All results come from Vivado simulation of the submitted RTL, scored by the provided MATLAB script, and were cross-checked with an independent bit-exact model.

A Repository structure and how to run

```
rtl/      cic_decimator.v  fir_decim_comp.v  task1_cic_top.v  task2_decim_top.v
tb/       tb_task1.v  tb_task2.v                (set `define TONE 10 / 500 / 990)
matlab/   fft_2.m  snr_*.m  run_all_snr.m  design_fir.m  fir_coeffs_q18.txt
results/  outputs/*.txt (simulation outputs)   *.png (PSD plots, responses)
```

1. In Vivado, add `rtl/*.v` as design sources, and `tb_task1.v` or `tb_task2.v` plus the three provided `.mem` files as simulation sources.
2. Set `TONE`, run the behavioural simulation, then `run all` (about 410 ms of simulated time).
3. Copy `cic2048_<TONE>.txt` or `fircomp_<TONE>.txt` from `<project>.sim/sim_1/behav/xsim/` to `matlab/`, then run `run_all_snr`.

B Verilog source

task1_cic_top.v

```
`timescale 1ns / 1ps
//=============================================================================
// task1_cic_top.v  -  TASK 1: standalone CIC decimator (OSR 2048)
//  din (2-bit, {-1,0,+1}) @ 4.096 MHz  ->  dout (20-bit signed) @ 2 kHz
//  CIC N=3, R=2048, M=1, output gain 3.5 (=7/2).
//  The input is captured on the falling clock edge: the provided testbench
//  updates 'bits' with a blocking assignment on posedge clk, which would
//  race with posedge-sampled logic.
//=============================================================================
// TASK 1: CIC N=3, R=2048, M=1, gain 3.5, 20-bit output
module task1_cic_top (
    input  wire clk, rst, en,
    input  wire signed [1:0]  din,
    output wire signed [19:0] dout,
    output wire dout_valid, clip
);
    reg signed [1:0] din_q;                 // negedge capture (fixes tb race)
    always @(negedge clk) din_q <= rst ? 2'sd0 : din;

    cic_decimator #(.N(3), .R(2048), .M(1), .IN_W(2), .OUT_W(20),
                    .GAIN_NUM(7), .GAIN_FRAC(1))
    u_cic (.clk(clk), .rst(rst), .en(en), .din(din_q),
           .dout(dout), .dout_valid(dout_valid), .clip(clip));
endmodule
```

task2_decim_top.v

```
`timescale 1ns / 1ps
//=============================================================================
// task2_decim_top.v  -  TASK 2: CIC + FIR droop compensator (OSR 2048)
//  din (2-bit) @ 4.096 MHz -> CIC (N=4, R=512) -> 24-bit @ 8 kHz
//                          -> 27-tap FIR, decimate by 4 -> dout 20-bit @ 2 kHz
//  Negedge input capture for the same reason as Task 1.
//=============================================================================
// TASK 2: CIC N=4, R=512 -> 24-bit -> 27-tap FIR comp, decimate 4 -> 20-bit
module task2_decim_top (
    input  wire clk, rst, en,
    input  wire signed [1:0]  din,
    output wire signed [23:0] cic_out,
    output wire cic_valid,
    output wire signed [19:0] dout,
    output wire dout_valid, clip
);
    wire clip_cic, clip_fir;
    assign clip = clip_cic | clip_fir;

    reg signed [1:0] din_q;                 // negedge capture (fixes tb race)
    always @(negedge clk) din_q <= rst ? 2'sd0 : din;

    cic_decimator #(.N(4), .R(512), .M(1), .IN_W(2), .OUT_W(24),
                    .GAIN_NUM(1), .GAIN_FRAC(0))
    u_cic (.clk(clk), .rst(rst), .en(en), .din(din_q),
           .dout(cic_out), .dout_valid(cic_valid), .clip(clip_cic));

    fir_decim_comp #(.IN_W(24), .OUT_W(20), .COEF_W(16), .FRAC(18), .ACC_W(48))
    u_fir (.clk(clk), .rst(rst), .din(cic_out), .din_valid(cic_valid),
           .dout(dout), .dout_valid(dout_valid), .clip(clip_fir));
endmodule
```

cic_decimator.v

```
`timescale 1ns / 1ps
//=============================================================================
// cic_decimator.v  -  Parameterised N-stage CIC decimator
// Digital Decimation Filter Design Challenge (Electronics Club, IIT Guwahati)
//
//  H(z) = [ (1 - z^-(R*M)) / (1 - z^-1) ]^N
//  * N integrators at the input rate, N combs at the decimated rate
//    (Hogenauer structure) - no multipliers in the filter path.
//  * All registers are BMAX = N*log2(R*M) + IN_W bits wide; two's-complement
//    wrap-around in the integrators is harmless because the combs only take
//    differences (result is exact modulo 2^BMAX).
//  * Combs are sequenced one stage per clock after each decimation tick.
//  * Output: constant gain GAIN_NUM/2^GAIN_FRAC, round-half-up to OUT_W bits,
//    saturate; 'clip' pulses if a sample saturated.
//  Used as  Task 1: N=3, R=2048, OUT_W=20, gain 7/2   (BMAX = 35)
//           Task 2: N=4, R=512,  OUT_W=24, gain 1     (BMAX = 38)
//=============================================================================
// N-stage CIC decimator, wrap-around arithmetic, round + saturate output
module cic_decimator #(
    parameter integer N = 4, R = 512, M = 1, IN_W = 2, OUT_W = 20,
    parameter integer GAIN_NUM = 1, GAIN_FRAC = 0   // gain = GAIN_NUM / 2^GAIN_FRAC
)(
    input  wire clk, rst, en,
    input  wire signed [IN_W-1:0]  din,
    output reg  signed [OUT_W-1:0] dout,
    output reg  dout_valid,
    output reg  clip
);
    localparam integer LOG2RM = $clog2(R*M);
    localparam integer BMAX   = N*LOG2RM + IN_W;           // Hogenauer bound
    localparam integer CNT_W  = (R > 1) ? $clog2(R) : 1;
    localparam integer GW     = BMAX + 9;
    localparam integer SH     = N*LOG2RM - (OUT_W-1) + GAIN_FRAC;

    // integrators
    reg signed [BMAX-1:0] integ [0:N-1];
    integer k;
    always @(posedge clk) begin
        if (rst) for (k = 0; k < N; k = k + 1) integ[k] <= 0;
        else if (en) begin
            integ[0] <= integ[0] + {{(BMAX-IN_W){din[IN_W-1]}}, din};
            for (k = 1; k < N; k = k + 1) integ[k] <= integ[k] + integ[k-1];
        end
    end

    // downsampler
    reg [CNT_W-1:0] cnt; reg tick;
    always @(posedge clk) begin
        if (rst) begin cnt <= 0; tick <= 0; end
        else begin
            tick <= 0;
            if (en) begin
                if (cnt == R-1) begin cnt <= 0; tick <= 1; end
                else cnt <= cnt + 1'b1;
            end
        end
    end

    // combs (one stage per clock after each tick)
    reg signed [BMAX-1:0] cval [0:N];
    reg signed [BMAX-1:0] dl   [0:N*M-1];
    reg [N:0] vs;
    integer s, j;
    always @(posedge clk) begin
        if (rst) begin
            vs <= 0;
            for (s = 0; s <= N; s = s + 1)   cval[s] <= 0;
            for (s = 0; s <  N*M; s = s + 1) dl[s]   <= 0;
        end else begin
            vs <= {vs[N-1:0], tick};
            if (tick) cval[0] <= integ[N-1];
            for (s = 1; s <= N; s = s + 1)
                if (vs[s-1]) begin
                    cval[s] <= cval[s-1] - dl[(s-1)*M + M-1];
                    for (j = M-1; j > 0; j = j - 1) dl[(s-1)*M + j] <= dl[(s-1)*M + j-1];
                    dl[(s-1)*M] <= cval[s-1];
                end
        end
    end

    // gain, round, saturate
    wire signed [GW-1:0] cext = {{(GW-BMAX){cval[N][BMAX-1]}}, cval[N]};
    wire signed [GW-1:0] prod = cext * GAIN_NUM;
    wire signed [GW-1:0] half = {{(GW-1){1'b0}}, 1'b1} <<< (SH-1);
```

```
    wire signed [GW-1:0] rnd  = (prod + half) >>> SH;
    wire signed [GW-1:0] MAXV = {{(GW-OUT_W+1){1'b0}}, {(OUT_W-1){1'b1}}};
    wire signed [GW-1:0] MINV = ~MAXV;
    always @(posedge clk) begin
        if (rst) begin dout <= 0; dout_valid <= 0; clip <= 0; end
        else begin
            dout_valid <= vs[N];
            clip <= 0;
            if (vs[N]) begin
                if      (rnd > MAXV) begin dout <= MAXV[OUT_W-1:0]; clip <= 1; end
                else if (rnd < MINV) begin dout <= MINV[OUT_W-1:0]; clip <= 1; end
                else                       dout <= rnd[OUT_W-1:0];
            end
        end
    end
endmodule
```

fir_decim_comp.v

```
`timescale 1ns / 1ps
//=============================================================================
// fir_decim_comp.v  -  27-tap CIC droop compensator + decimate-by-4
// Digital Decimation Filter Design Challenge (Electronics Club, IIT Guwahati)
//
//  * Runs at the CIC output rate (8 kHz); one output per 4 inputs (2 kHz).
//  * Response: |H(f)| = 1/|H_CIC(f)| on 0..1000 Hz (inverse sinc^4, R=512),
//    stop band 2..4 kHz; least-squares design (firls), stop-band weight 10.
//    Combined CIC x FIR ripple 0..990 Hz = 0.043 dB, alias rejection ~59 dB.
//  * Coefficients: signed Q18 in 16-bit words. Their sum (57345 = 0.21875*2^18)
//    folds in the overall gain 3.5 and the 24-bit -> 20-bit scaling (1/16).
//  * Linear phase -> symmetric taps -> pre-adder x[i]+x[26-i]; 14 unique
//    products computed on ONE multiplier over 14 clocks (2048 clocks are
//    available per output sample).
//  * 48-bit accumulator; output round-half-up (>>>18) and saturate to 20 bits.
//=============================================================================
// 27-tap symmetric FIR CIC-droop compensator, decimate by 4, one shared multiplier
module fir_decim_comp #(
    parameter integer IN_W = 24, OUT_W = 20, COEF_W = 16, FRAC = 18, ACC_W = 48
)(
    input  wire clk, rst,
    input  wire signed [IN_W-1:0] din,
    input  wire din_valid,
    output reg  signed [OUT_W-1:0] dout,
    output reg  dout_valid,
    output reg  clip
);
    localparam integer K = 27, NU = 14, DEC = 4;

    function signed [COEF_W-1:0] coef;     // Q18 coefficients (first half)
        input [3:0] i;
        case (i)
            4'd0: coef =    75;  4'd1: coef =   165;  4'd2: coef =    14;
            4'd3: coef =  -447;  4'd4: coef =  -631;  4'd5: coef =   213;
            4'd6: coef =  1584;  4'd7: coef =  1447;  4'd8: coef = -1396;
            4'd9: coef = -4472;  4'd10: coef = -2704; 4'd11: coef = 6142;
            4'd12: coef = 17397; 4'd13: coef = 22571;   // centre tap
            default: coef = 0;
        endcase
    endfunction

    // delay line + decimation phase
    reg signed [IN_W-1:0] x [0:K-1];
    reg [1:0] ph; reg start;
    integer t;
    always @(posedge clk) begin
        if (rst) begin for (t = 0; t < K; t = t + 1) x[t] <= 0; ph <= 0; start <= 0; end
        else begin
            start <= 0;
            if (din_valid) begin
                x[0] <= din;
                for (t = 1; t < K; t = t + 1) x[t] <= x[t-1];
                if (ph == DEC-1) begin ph <= 0; start <= 1; end
                else ph <= ph + 1'b1;
            end
        end
    end

    // serial MAC with pre-adder
    reg [3:0] idx; reg busy, mvld, mlast, done;
    reg signed [IN_W:0]     pre;
    reg signed [COEF_W-1:0] cq;
    reg signed [ACC_W-1:0]  acc;
    wire [4:0] mirror = K-1 - idx;
    wire signed [IN_W:0] xa = {x[idx][IN_W-1], x[idx]};
    wire signed [IN_W:0] xb = {x[mirror][IN_W-1], x[mirror]};
    wire signed [IN_W+COEF_W:0] prodw = pre * cq;
    always @(posedge clk) begin
        if (rst) begin idx <= 0; busy <= 0; mvld <= 0; mlast <= 0; pre <= 0; cq <= 0; acc <= 0; done <= 0; end
        else begin
            done <= 0;
            if (start) begin busy <= 1; idx <= 0; end
            else if (busy) begin idx <= idx + 1'b1; if (idx == NU-1) busy <= 0; end
            mvld  <= busy;
            mlast <= busy && (idx == NU-1);
            if (busy) begin
                pre <= (idx == NU-1) ? xa : (xa + xb);
                cq  <= coef(idx);
            end
            if (start)     acc <= 0;
            else if (mvld) acc <= acc + {{(ACC_W-IN_W-COEF_W-1){prodw[IN_W+COEF_W]}}, prodw};
            if (mvld && mlast) done <= 1;
        end
    end
```

```
    // round, saturate
    wire signed [ACC_W-1:0] half = {{(ACC_W-1){1'b0}}, 1'b1} <<< (FRAC-1);
    wire signed [ACC_W-1:0] rnd  = (acc + half) >>> FRAC;
    wire signed [ACC_W-1:0] MAXV = {{(ACC_W-OUT_W+1){1'b0}}, {(OUT_W-1){1'b1}}};
    wire signed [ACC_W-1:0] MINV = ~MAXV;
    always @(posedge clk) begin
        if (rst) begin dout <= 0; dout_valid <= 0; clip <= 0; end
        else begin
            dout_valid <= done;
            clip <= 0;
            if (done) begin
                if      (rnd > MAXV) begin dout <= MAXV[OUT_W-1:0]; clip <= 1; end
                else if (rnd < MINV) begin dout <= MINV[OUT_W-1:0]; clip <= 1; end
                else                       dout <= rnd[OUT_W-1:0];
            end
        end
    end
endmodule
```